Recurrent Neural Networks

Applied Deep Learning — Chapter 9

Yin Yang

Foundation Books Project

Roadmap

Sequence Data and the IMDB Running Example

The Simple Elman RNN

Unrolling Through Time and BPTT

Padding, Masks, Collation, Bucketing

Why Simple RNNs Struggle

LSTM, GRU, and the Gating Pattern

Regularization and Normalization

Practical IMDB Recipe and AWD-LSTM

Summary

Sequence Data and the IMDB Running Example

Why Order Matters

Text, audio, weather, medical visits: each example is an ordered tuple $(x_1, \ldots, x_T)$.

- "not good" $\neq$ "good, not great" — same words, different order.
- Flattening hides the structure "one position follows another."
- Feedforward bag-of-words classifiers tend to lose this evidence.

Running example: IMDB binary sentiment (positive vs. negative) on tokenized reviews.

Sequence Task Shapes

Many-to-one read a sequence, emit one label *(IMDB sentiment, this chapter)*.

Many-to-many one label per token (POS tagging).

Sequence-to-sequence output is also a sequence, possibly different length (translation).

One-to-many one prompt, many outputs (music generation).

Common feature: an example is an *ordered record*, not a fixed unordered vector.

IMDB Tensor Shapes

After embedding, a padded mini-batch has shape

$$(B, T_{\max}, D) = (32, 200, 64).$$

Three axes:

- **Batch**: which review.
- **Time/position**: which token in the review.
- **Features**: embedding coordinates at one position.

An RNN scans along the time axis, carrying a hidden state from position to position.

The Simple Elman RNN

The Elman Cell

Inputs $x_t \in \mathbb{R}^D$; hidden state $h_t \in \mathbb{R}^H$; hidden width H chosen. With $h_0 = 0$,

$$a_t = W_{xh} x_t + W_{hh} h_{t-1} + b_h, \qquad h_t = \phi(a_t),$$

with $\phi = \tanh$ usually.

Many-to-one classifier head from final valid state h_T:

$$z = w_y^\top h_T + b_y.$$

Weight sharing: the same W_{xh}, W_{hh} are reused at every position, so parameter count is independent of T.

Shape Check

Let $D = 64$, $H = 128$, batch first $(B, T, D) = (32, 200, 64)$.

- $W_{xh} \in \mathbb{R}^{128 \times 64}$
- $W_{hh} \in \mathbb{R}^{128 \times 128}$
- $b_h \in \mathbb{R}^{128}$
- Layer returns all states: $(32, 200, 128)$.
- Returns final state: $(32, 128)$.
- Sentiment logit: $z \in \mathbb{R}$.

Memory is *learned*: h_t is a vector-valued function of the prefix $(x_1, \ldots, x_t)$, not a tape.

Scalar Walk-through

Take $D = H = 1$, $h_0 = 0$, $\phi = \tanh$, $w_x = 1.2$, $w_h = 0.5$, $b = 0$. Tokens encode as $x_1 = 0.6$, $x_2 = -0.4$, $x_3 = 0.8$.

$$h_1 = \tanh(0.72) \approx 0.617,$$

$$h_2 = \tanh(1.2(-0.4) + 0.5(0.617)) \approx -0.170,$$

$$h_3 = \tanh(1.2(0.8) + 0.5(-0.170)) \approx 0.704.$$

Output at step 3 is not a function of x_3 alone — earlier tokens enter through h_2.

Stacking Recurrent Layers

Two depths in a stacked RNN:

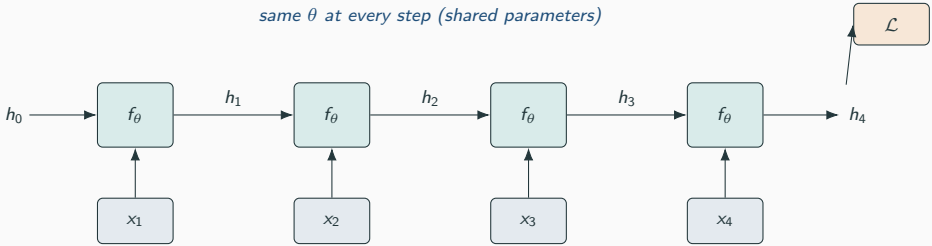
$$h_t^{(1)} = f_{\theta_1}(x_t, h_{t-1}^{(1)}), \qquad h_t^{(\ell)} = f_{\theta_\ell}(h_t^{(\ell-1)}, h_{t-1}^{(\ell)}).$$

- **Time depth** runs horizontally (sequence positions).
- **Layer depth** runs vertically (lower $\rightarrow$ higher layer).
- Lower layers must return the whole sequence; the top layer may return either the full sequence or only the final valid state.

Stacking is a later experiment, not a substitute for first picking the cell.

Unrolling Through Time and BPTT

Unrolling



- Same parameter set θ in every copy.
- Standard backpropagation applies to the unrolled graph (BPTT).

Shared-Parameter Gradient

Loss from the final state contributes to W_{hh} at every step:

$$\frac{\partial \mathcal{L}}{\partial W_{hh}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}}{\partial h_t} \frac{\partial h_t}{\partial W_{hh}}.$$

- $\partial \mathcal{L}/\partial h_t$ chains through later states $h_{t+1}, \ldots, h_T$.
- Many recurrent Jacobians multiplied along the temporal path.
- Memory cost: training stores intermediate states for the backward pass.

Long sequences cost more memory and more time.

Truncated BPTT and Dynamic RNNs

Truncated BPTT. Backpropagate through shorter windows.

- Cuts memory; limits long-range credit assignment.
- Less critical for IMDB (finite reviews) than choosing $T_{\max}$.

"Dynamic RNN" = the recurrent loop runs at the actual length, with masks or packed sequences for padded positions. Padding is not data. Final-state classifiers must use the *last valid* state, not the state after padding.

Padding, Masks, Collation, Bucketing

Padding a Mini-Batch

Three reviews, max length 4, pad token ID 0:

review	padded IDs				mask			
1	12	48	23	77	1	1	1	1
2	95	77	0	0	1	1	0	0
3	310	44	822	0	1	1	1	0

Padded tensor: $(3, 4)$. Mask: same first two dims — "which positions carry real tokens."

Padding Waste and Bucketing

Padding waste = (slots − real tokens)/slots. Lengths $(600, 40, 38, 37)$ padded to 600: 2400 slots, 715 real $\Rightarrow \approx 70\%$ waste. **Bucketing:** group examples of similar length before batching.

- Six lengths $(52, 50, 49, 6, 5, 4)$.
- Random batches: $\sim 45.8\%$ wasted.
- Length-bucketed batches: $\sim 4.6\%$ wasted.

Tradeoff: full sort kills randomness; shuffle within buckets.

Collation Checklist

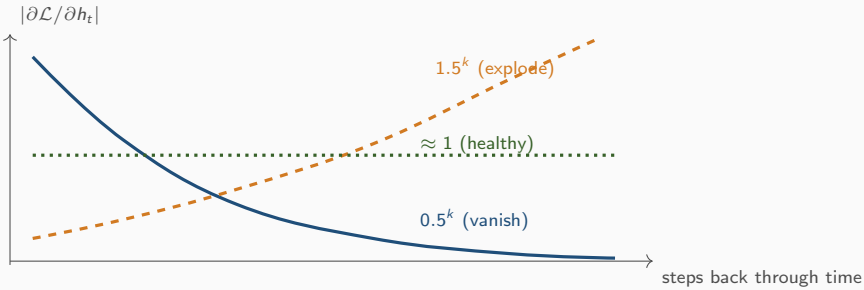
Collation builds a mini-batch from individual examples.

- Find lengths; pick padding length.
- Pad token IDs and labels to a rectangle.
- Return either a *mask* or a *length vector*.
- Keras: `pad_sequences` + `mask_zero=True`.
- PyTorch: collate function on `DataLoader`, optionally packed sequences.

Model must receive both the rectangular tensor and enough information to distinguish tokens from padding.

Why Simple RNNs Struggle

Vanishing and Exploding Gradients



Over 10 steps: $0.5^{10} \approx 10^{-3}$, $1.5^{10} \approx 58$. Real RNNs: matrices $\times$ nonlinearity Jacobians — same product structure.

tanh saturates far from zero, so derivatives shrink precisely when state magnitudes grow.

Engineering Responses

Gradient clipping limits the gradient norm before the optimizer step. Good guard against *exploding* gradients; does *not* fix vanishing.

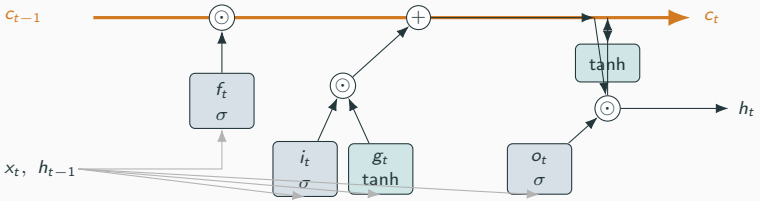
Gated cells LSTM and GRU give a more direct path for preserving state.

Modest learning rates and early stopping supplement clipping.

Avoid heroic tuning of a weak baseline. If a simple RNN does not beat an averaged-embedding baseline, switch to GRU or LSTM rather than stacking dropout and layers.

LSTM, GRU, and the Gating Pattern

The LSTM Cell



$$c_t = f_t \odot c_{t-1} + i_t \odot g_t, \qquad h_t = o_t \odot \tanh(c_t).$$

Additive cell-state highway carries memory; gates control read/write.

LSTM Numerical Sketch

Suppose $c_{t-1} = 1.0$, $f_t = 0.9$, $i_t = 0.2$, $g_t = 0.5$, $o_t = 0.8$.

$$c_t = 0.9(1.0) + 0.2(0.5) = 1.0,$$

$$h_t = 0.8 \tanh(1.0) \approx 0.610.$$

- Forget gate $f_t \approx 1$: retain old memory.
- Input gate i_t small: block new writes.
- Output gate o_t: how much of c_t leaks into h_t.

"Highway link": information moves through time by repeated elementwise additions and scalings.

Peephole LSTM and GRU

Peephole LSTM: gates also see the cell state, e.g. $f_t \leftarrow f_t + p_f \odot c_{t-1}$ inside its sigmoid.

GRU: one state vector, update z_t and reset r_t gates, candidate n_t,

$$h_t = (1 - z_t) \odot n_t + z_t \odot h_{t-1}.$$

Linear interpolation between previous state and candidate. No universal winner. Fix everything else, change only the cell, then compare validation accuracy, time, and parameter count.

Parameter Counts

Per recurrent group with input D, hidden H:

$$\text{params} = H(D + H) + H.$$

With $D = 32$, $H = 64$:

Cell	Param count	
Simple RNN	6,208	1 group
GRU	18,624	3 groups
LSTM	24,832	4 groups

Cost grows with widths, layers, and embedding dimensions.

Gating Beyond Recurrent Cells

Pattern: output $=$ signal $\odot$ gate, with gate in $[0, 1]$.

Recurrent gate sigmoid$(W_x x_t + W_h h_{t-1} + b)$: temporal, context-dependent.

Squeeze-and-excitation global-average channel summary $\rightarrow$ sigmoid $\rightarrow$ multiply each channel; not recurrent.

Swish $a \cdot \text{sigmoid}(\beta a)$: self-gated activation.

ReLU hard self-gate: $a \cdot \mathbf{1}[a > 0]$.

"Gate" names a design pattern, not one specific layer.

Regularization and Normalization

Dropout for RNNs

Where the mask lives matters.

- **Vertical / between-layer** dropout: usually safe.
- **Horizontal / recurrent** dropout: fresh mask per step can break information transport.
- **Variational / locked** dropout: *same* mask across steps, so a coordinate is consistently retained or dropped.

Inverted dropout, $h_t = (2, 4, 6)$, mask $(1, 0, 1)$, keep prob 0.5:

$$\frac{m \odot h_t}{q} = (4, 0, 12).$$

Train/eval mode discipline still applies.

Normalization in RNNs

Batch normalization is awkward in recurrent state updates: statistics must define which time steps and which (un-padded) positions contribute.

Layer normalization is more natural: stats over features within one example at one position.

$h = (1, 3, 5)$: $\mu = 3$, $\sigma^2 = 8/3$,

$$\tilde{h} \approx (-1.225, 0, 1.225).$$

Beginner rule: do not insert batchnorm into a recurrent state path unless the library provides a documented variant.

Practical IMDB Recipe and AWD-LSTM

Run Plan: One Variable at a Time

Hold split, vocabulary, $T_{\max}$, batch size, optimizer, epochs constant. Change only the cell.

Run	Model	Touches test?
A	Avg. embeddings	no
B	Simple RNN	no
C	GRU	no
D	LSTM	no

Lock the test set until *one* model has been selected on validation evidence.

Measured IMDB Numbers

Run	Pad ratio	Val. acc.	Val. loss	Time
Avg. emb., len. 200	0.194	0.8504	0.363	8.0 s
Simple RNN, len. 200	0.194	0.8378	0.399	181.8 s
GRU, len. 200	0.194	0.8590	0.343	223.7 s
LSTM, len. 200	0.194	0.8574	0.382	220.8 s
GRU, len. 80	0.020	0.8028	0.432	109.1 s
GRU, len. 400	0.471	0.8718	0.313	420.4 s
GRU + rec. dropout 0.2	0.194	0.8328	0.386	395.7 s

Selected GRU/len. 400 reached 0.8658 on the locked test set — about 1.9× runtime, nearly half the slots padding.

Tradeoffs Made Visible

- Average embeddings is a strong, almost-free baseline.
- Simple RNN: educational, slower, and weaker than the baseline here.
- GRU and LSTM: similar parameter count, similar accuracy on this split.
- Length 80: cheap but truncates evidence; 400: pricier but better.
- Recurrent dropout slowed training and did not help validation.

A name is not evidence. Quality, speed, and cost are reported together.

Minimal Keras Skeleton

```
inputs  = keras.Input(shape=(max_length,), dtype="int32")
x = layers.Embedding(num_words, embedding_dim,
                     mask_zero=True)(inputs)
x = layers.LSTM(hidden_size)(x)
outputs = layers.Dense(1)(x)
model = keras.Model(inputs, outputs)
```

`mask_zero=True` carries the padding mask into the recurrent layer so the final state ignores padded positions. Token ID 0 is reserved for padding only.

AWD-LSTM Recipe in One Slide

fast.ai's `AWD_LSTM`: ASGD Weight-Dropped LSTM (Merity et al.). Coordinated bundle of regularizers:

- embedding dropout
- locked dropout on sequence activations
- between-layer dropout
- weight dropout on the hidden-to-hidden matrices
- gradient clipping; staged fine-tuning for transfer learning

Knobs: `embed_p`, `input_p`, `hidden_p`, `weight_p`, `drop_mult`. A `drop_mult` change moves several knobs at once — not evidence about one location.

Accuracy and Speed Checklist

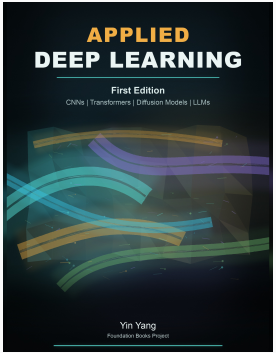
- Choose $T_{\max}$ from the actual length distribution.
- Measure padding ratio per batch.
- Use masks or packed sequences; final-state correctness.
- Apply gradient clipping by default for recurrent models.
- Save best validation epoch / use early stopping.
- Prefer layer normalization over casual batchnorm in recurrent paths.
- Compare cells under one budget; report accuracy + runtime + params.

Summary

Summary

- RNN = repeatedly applies one shared state-update along an ordered sequence; loop $\rightarrow$ unrolled graph $\rightarrow$ BPTT.
- Simple Elman cell exposes vanishing/exploding gradients; clipping helps the latter, gated cells help the former.
- LSTM and GRU add additive state paths and learned gates; gating is a general design pattern.
- Sequence pipelines add risk: padding, masks, collation, bucketing, dropout placement, normalization axes.
- Strong recurrent results come from *recipes* (e.g. AWD-LSTM), not from a single cell change.
- Report accuracy and speed together; lock the test set.

Next: attention and Transformers — shorten the information path so every output can read directly from any sequence position.



Applied Deep Learning

Chapter overview slides for the book by Yin Yang.

Book page	`foundation-books.github.io/applied-deep-learning`
Code	`github.com/foundation-books/applied-deep-learning-code`
Print	`amazon.com/dp/B0GZF7QRSH`
Kindle	`amazon.com/dp/B0GZF9221X`
License	CC BY-NC-ND 4.0

These free overview slides may be shared non-commercially with attribution and without modifications. Full explanations, exercises, and companion-code context are in the book and linked repository.